Industrial Training / Institutional Project

Final Project Report Summary

Academic Year: 2025–26

Project Title : **Authentication & Authorization Module for College ERP System**

Seat No.:2359

Student Name**: Swaraj Bhagat**

Class: S.Y.M.Sc .(CS)

College Name: **PVG's College of Science & Commerce, Pune**

Project Guide Name: **Mrs Rekha Joshi**

Mentor / Technical Guide **:- Mr Rajesh Darak**

Institute Name : PVG College of Science & Commerce

Project Introduction :-

The College ERP system is designed to manage academic and administrative activities such as student data, attendance, fees, hostel, and transport in a centralized platform. The main focus of our project is the **Authentication and Authorization Module**, which ensures secure access to the system using modern security techniques. The system is developed using React for the frontend, FastAPI for the backend, and PostgreSQL as the database, making it scalable and efficient.The problem addressed in this project is the lack of secure and structured access control in traditional college systems, which can lead to unauthorized access and data breaches. To solve this, we implemented **JWT-based authentication**, where each user is verified through secure login credentials, and a token is generated for session management. Passwords are securely stored using bcrypt hashing, ensuring data protection.The methodology includes Role-Based Access Control (RBAC), where users are assigned roles such as Guest, Student, or Admin, and each role has limited permissions. By default, new users are assigned a **Guest role** to follow the principle of least privilege. The system also tracks token activity and includes audit fields like created_at and updated_at for better monitoring.The key outcome of this project is a **secure, scalable, and modular authentication system** that enhances data

security, improves user management, and provides a strong foundation for future ERP development.

Institute Profile

The College ERP Authentication and Authorization Module is being developed for PVG's College of Science & Commerce, Pune, which is affiliated with Savitribai Phule Pune University and known for providing quality education in science and commerce fields. This project is a part of the M.Sc. Computer Science curriculum for the academic year 2025–26. The main aim of this module is to provide secure login and controlled access for students, teachers, and administrative users within the college ERP system. The module owner is Rekha Joshi Ma'am, who provided guidance regarding functional requirements and system flow. Rajesh Darak Sir is our mentor and technical guide, whose valuable suggestions, support, and technical knowledge helped us throughout the project. The system is developed using React for frontend, FastAPI for backend, and PostgreSQL for database management, with JWT authentication and bcrypt password hashing to ensure security and proper role-based access control.

System Objective:

The main objective of the College ERP Authentication and Authorization Module is to provide a secure, scalable, and role-based access system for students, faculty, and administrative users. The system ensures that only verified users can log in and access the information relevant to their roles using Role-Based Access Control (RBAC). It uses JWT for secure authentication and session management, while bcrypt is used for password hashing to protect user credentials. New users are assigned a default Guest role with minimum permissions to improve security. The system also includes token tracking, audit fields such as created_at and updated_at, and secure API validation. Developed using React, FastAPI, and PostgreSQL, the project aims to replace manual and less secure systems with a centralized and efficient solution. It also supports future integration with other

ERP modules like attendance, fees, exams, hostel, and transport, creating a strong foundation for a complete and secure College ERP system.

Methodology:

To address the security and management challenges in traditional college systems, we developed a secure Authentication and Authorization Module for the College ERP system. The backend is built using FastAPI for high performance and efficient API handling, while the frontend is developed using React to provide a simple and user-friendly interface. PostgreSQL is used as the database for storing structured user, role, and token information. The system uses JWT (JSON Web Token) for secure authentication and stateless session management. During login, user credentials are verified using bcrypt password hashing, and a secure token containing user ID, role, and expiration time is generated. This token is used for validating all future API requests. The system follows Role-Based Access Control (RBAC), where each user is assigned a specific role such as Guest, Student, or Admin. New users are assigned the Guest role by default to ensure minimal access and better security. Token tracking and audit fields like created_at and updated_at improve transparency and help monitor user activity. This module reduces unauthorized access, improves data protection, and provides a scalable foundation for future ERP modules like attendance, fees, exams, hostel, and transport, making it a strong base for a complete College ERP system.

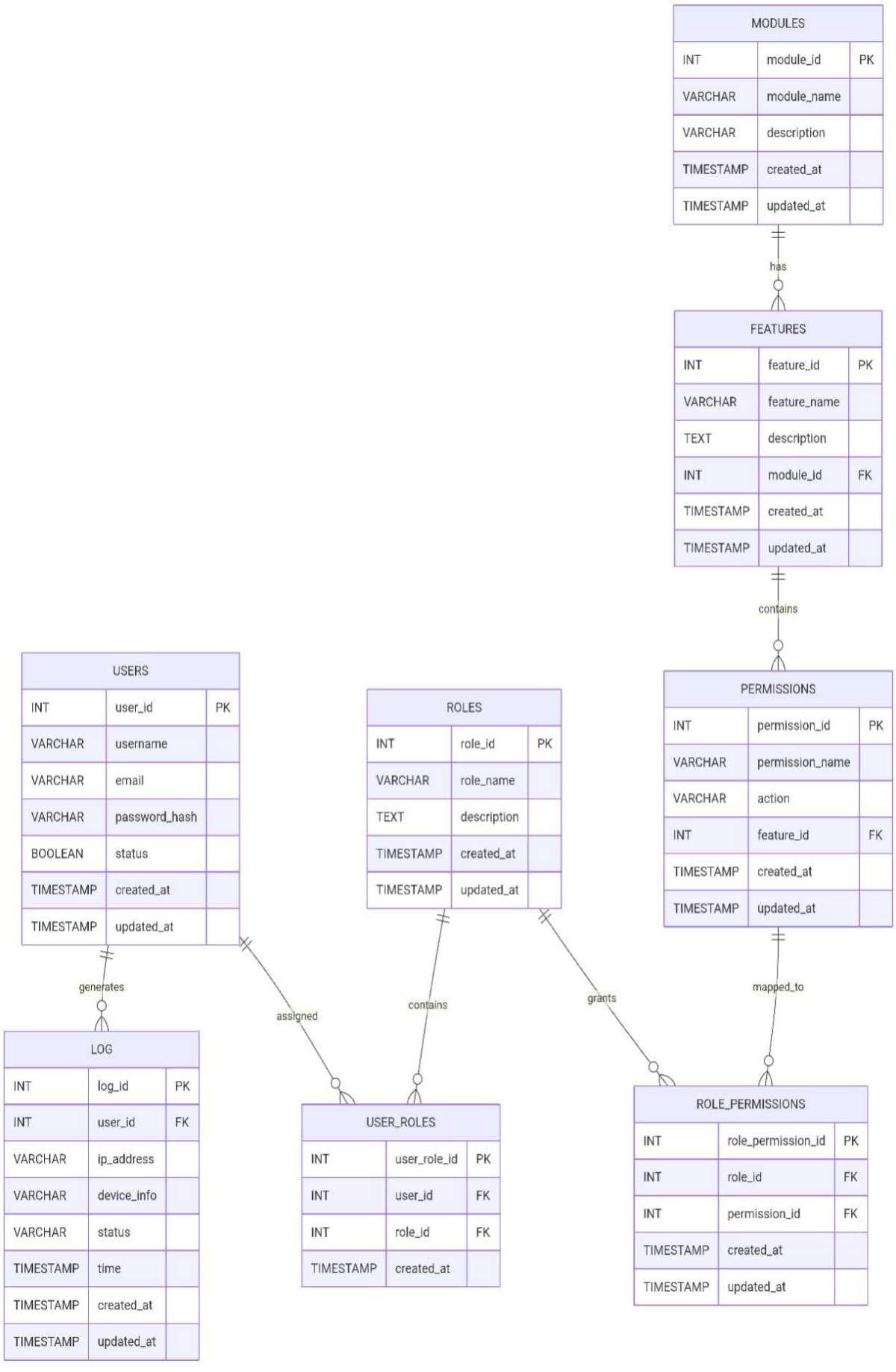
Requirement Analysis

The Requirement Analysis of the College ERP Authentication and Authorization Module includes both functional and non-functional requirements. The functional requirements explain what the system should do. The system must allow users to register and log in securely using a username and password. Passwords should be stored safely using bcrypt hashing so that they cannot be seen directly in the

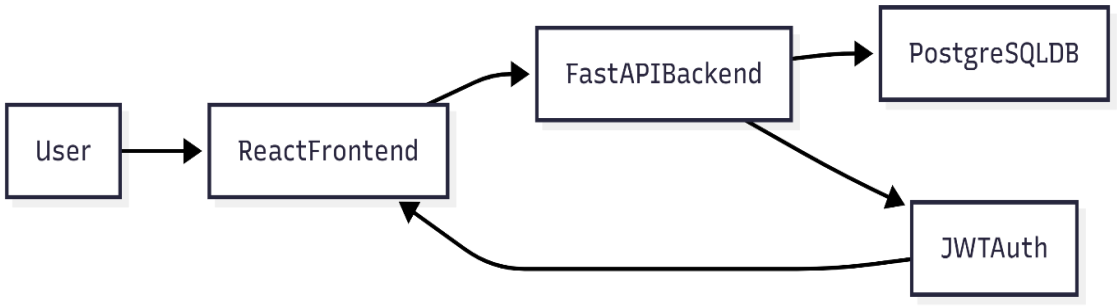
database. After successful login, the system should generate a JWT token for secure session management. This token helps in checking the user's identity for future requests without logging in again and again. The system should also support role assignment such as Guest, Student, Teacher, or Admin, and users should only access the data allowed for their role using Role-Based Access Control (RBAC). It should include APIs for signup, login, token verification, and protected routes. The system must also store token details and user session information in the database and show a welcome message after login. The non-functional requirements focus on system quality. The system should respond quickly and handle multiple users smoothly without slowing down. The backend should be lightweight and scalable for future growth. Security is very important, so passwords must be encrypted, JWT tokens should have expiry time, and unauthorized access must be prevented. Logout and token expiration should work properly for better protection. The system should also keep audit details like created time and updated time for tracking changes. Usability is also important, so the interface should be simple, clean, and easy to use for all users, even for non-technical people. The signup and login process should require only a few simple steps, and error messages should be clear so users can easily understand and correct mistakes. Overall, the system should be secure, fast, and user-friendly.

System Design

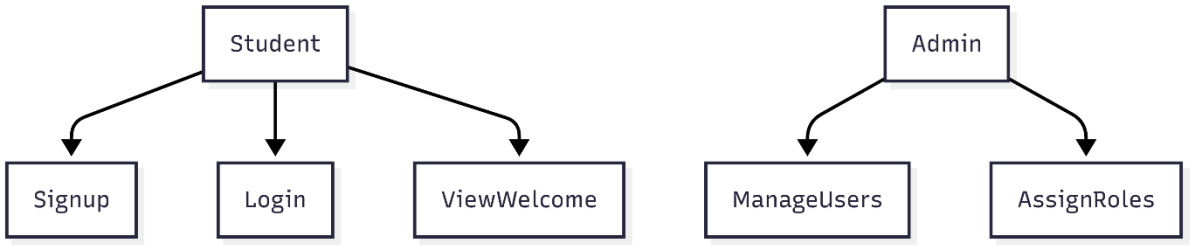
- **Database Design (ER Diagram / Tables)**


MODULES
INT module_id PK
VARCHAR module_name
VARCHAR description
TIMESTAMP created_at
TIMESTAMP updated_at
has
FEATURES
INT feature_id PK
VARCHAR feature_name
TEXT description
INT module_id FK
TIMESTAMP created_at
TIMESTAMP updated_at
contains
PERMISSIONS
INT permission_id PK
VARCHAR permission_name
VARCHAR action
INT feature_id FK
TIMESTAMP created_at
TIMESTAMP updated_at
mapped_to
USERS
INT user_id PK
VARCHAR username
VARCHAR email
VARCHAR password_hash
BOOLEAN status
TIMESTAMP created_at
TIMESTAMP updated_at
generates
assigned
ROLES
INT role_id PK
VARCHAR role_name
TEXT description
TIMESTAMP created_at
TIMESTAMP updated_at
contains
grants
LOG
INT log_id PK
INT user_id FK
VARCHAR ip_address
VARCHAR device_info
VARCHAR status
TIMESTAMP time
TIMESTAMP created_at
TIMESTAMP updated_at
USER_ROLES
INT user_role_id PK
INT user_id FK
INT role_id FK
TIMESTAMP created_at
ROLE_PERMISSIONS
INT role_permission_id PK
INT role_id FK
INT permission_id FK
TIMESTAMP created_at
TIMESTAMP updated_at

- ## Architecture Diagram
- The system follows a **three-tier architecture** consisting of frontend, backend, and database layers. The frontend is developed using React, which handles user interaction and sends API requests. The backend is built using FastAPI, which processes requests, performs authentication, and manages business logic.

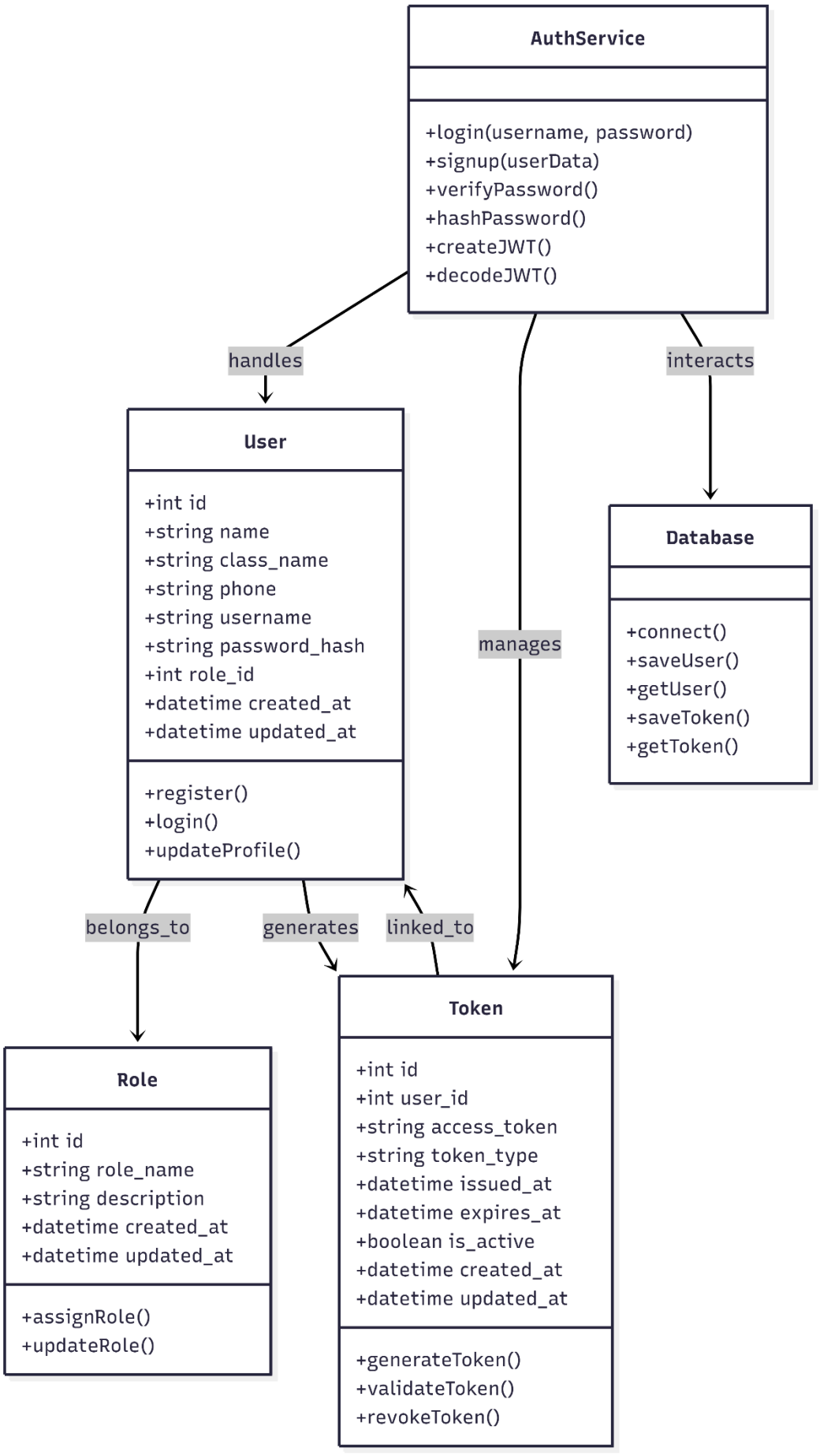


- ## UML Diagrams (Use Case, Class, Sequence, etc.)
- **1) Use Case :-**
- The use case diagram shows how different users interact with the system. In this project, the primary user is the student who can register, log in, and view the welcome page. Admin users can manage roles and users. The system handles authentication and authorization processes.



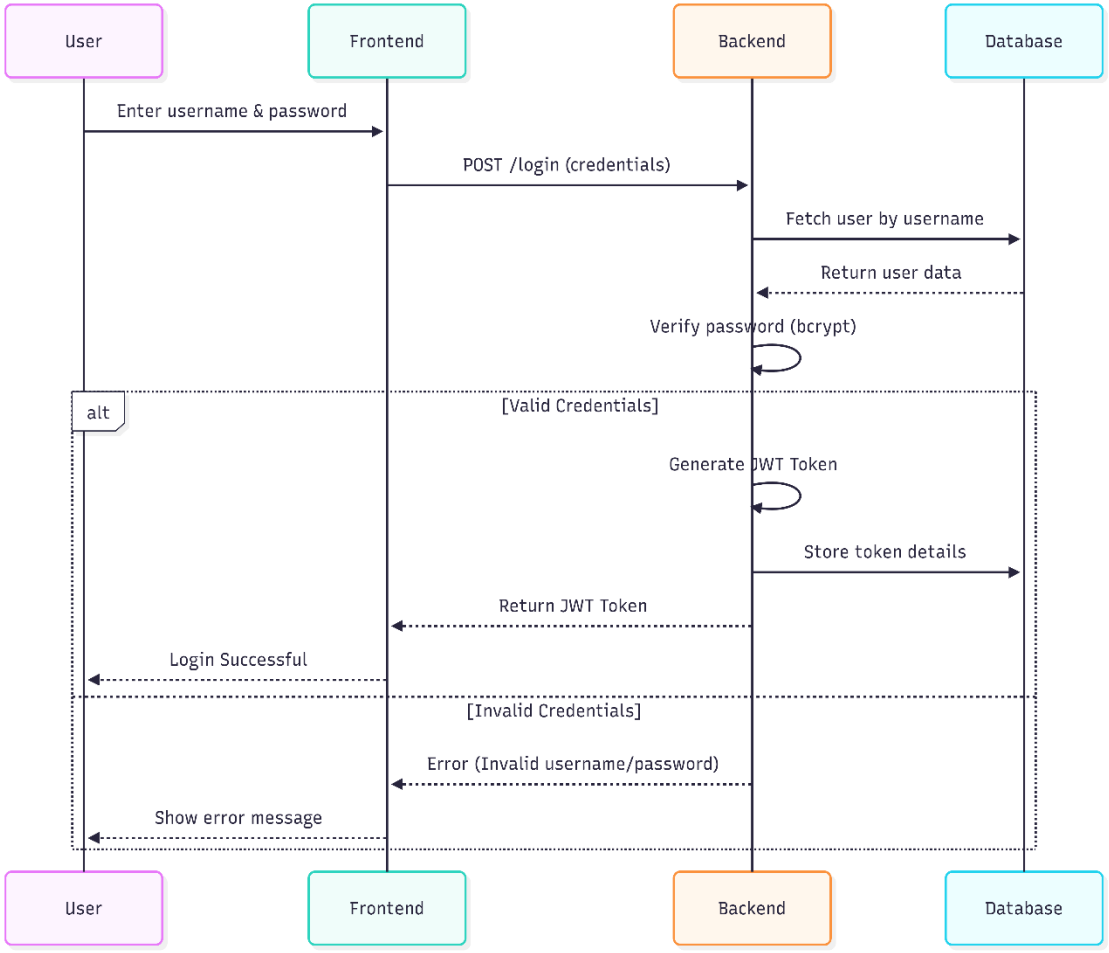
2) Class Diagram

The class diagram represents the overall structure of the system and shows how different components interact with each other. It mainly consists of core entities such as User, Role, and Token, along with supporting classes like AuthService and Database, which help in managing authentication logic and data storage.The User class stores all the essential details of a user, including personal information such as name, class, phone number, and login credentials like username and pass


AuthService
+login(username, password)
+signup(userData)
+verifyPassword()
+hashPassword()
+createJWT()
+decodeJWT()
handles
manages
interacts
User
+int id
+string name
+string class_name
+string phone
+string username
+string password_hash
+int role_id
+datetime created_at
+datetime updated_at
+register()
+login()
+updateProfile()
Database
+connect()
+saveUser()
+getUser()
+saveToken()
+getToken()
belongs_to
generates
linked_to
Role
+int id
+string role_name
+string description
+datetime created_at
+datetime updated_at
+assignRole()
+updateRole()
Token
+int id
+int user_id
+string access_token
+string token_type
+datetime issued_at
+datetime expires_at
+boolean is_active
+datetime created_at
+datetime updated_at
+generateToken()
+validateToken()
+revokeToken()

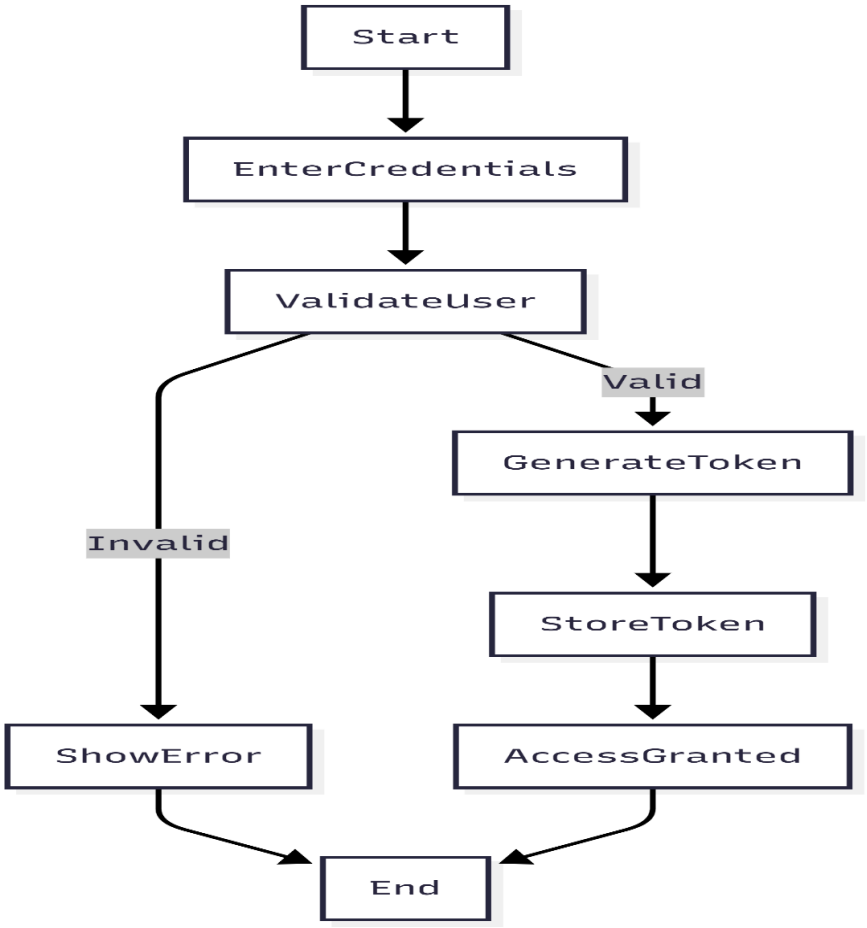
3) Sequence Diagram

The sequence diagram shows the step-by-step flow of the user login process in the College ERP system. First, the user enters the username and password on the React frontend. The frontend sends the login request to the FastAPI backend through an API call. The backend then checks the PostgreSQL database to verify whether the user exists. It retrieves the stored user details and validates the password using bcrypt hashing. If the credentials are correct, the backend generates a JWT token with user information and expiry time. The token is stored in the database for session tracking and security. Finally, the backend sends the token and success response back to the frontend, and the user is redirected to the welcome page. This process ensures secure authentication and proper session management.



4) Activity Diagram

The activity diagram illustrates the step-by-step workflow of the authentication process in the system. It begins when the user enters their login credentials on the interface. The system then validates the entered details by checking them against the stored data in the database. If the credentials are correct, the system generates a JWT token and stores it for session management. After successful token generation, access is granted to the user. If the credentials are invalid, an error message is displayed and the process ends without granting access



5) User Interface Design

The user interface is designed to be simple, clean, and easy to use, allowing even non-technical users to interact with the system without difficulty. It focuses on providing a smooth experience while performing basic authentication tasks. The system includes three main pages: Signup, Login, and Welcome. The Signup page allows users to enter their details with proper validation, while the Login page securely verifies user credentials and generates a JWT token on successful authentication. In case of invalid input, clear error messages are displayed. After login, the Welcome page shows a personalized message to the user. Overall, the interface is minimal, responsive, and ensures easy navigation across devices.

Development & Implementation

(Tools & Technologies Used, Modules Description, Screenshots)

• Software Specifications

The system is developed using modern web technologies to ensure performance, scalability, and security. The frontend is built using **React**, which provides a responsive and interactive user interface. The backend is developed using **FastAPI**, a high-performance Python framework that supports asynchronous programming and fast API execution.

For database management, **PostgreSQL** is used as it provides strong support for relational data and ensures data consistency. Authentication is implemented using **JSON Web Tokens (JWT)** for secure and stateless session handling. Passwords are encrypted using **bcrypt hashing** to enhance security.The system is containerized using **Docker and Docker Compose**, which simplifies deployment and ensures that the application runs consistently across different environments. The development is done using tools like Visual Studio Code and Git for version control.

• Hardware Requirements

The system does not require high-end hardware and can run efficiently on standard systems. The minimum hardware requirements are:

- **Processor:** Intel i3 or above
- **RAM:** 8 GB or higher recommended for smooth performance
- **Storage:** 20 GB free disk space
- **Operating System:** Windows, Linux, or macOS
- **Internet:** Required for installation and dependency management

These requirements ensure that the system can be easily developed and tested on commonly available machines.

• Tools & Technologies Used

The project uses a combination of modern tools and technologies:

- **Frontend:** React (for UI development)
- **Backend:** FastAPI (for API development)
- **Database:** PostgreSQL (for structured data storage)
- **Authentication:** JWT (for secure login and session management)

- **Security:** bcrypt (for password hashing)
- **Containerization:** Docker & Docker Compose (for deployment)
- **Version Control:** Git & GitHub
- **Editor:** Visual Studio Code

These technologies are chosen for their performance, scalability, and ease of integration.

• Module Description

The project mainly focuses on the **Authentication and Authorization Module** of the College ERP system.

1. Signup

This module allows new users to register by entering their details such as name, class, phone number, username, and password. The password is securely hashed using bcrypt before storing it in the database. By default, the user is assigned a **Guest role** to ensure minimal access.

2. Login

The login module verifies user credentials. When a user enters their username and password, the system checks the data against the database. If the credentials are valid, a **JWT token** is generated and returned to the user. This token is used for authentication in further requests.

3. Token Management

This module handles the creation, validation, and storage of JWT tokens. Each token contains user details, role information, and expiration time. Tokens are stored in the database to track user sessions and enhance security.

4. Authorization (RBAC)

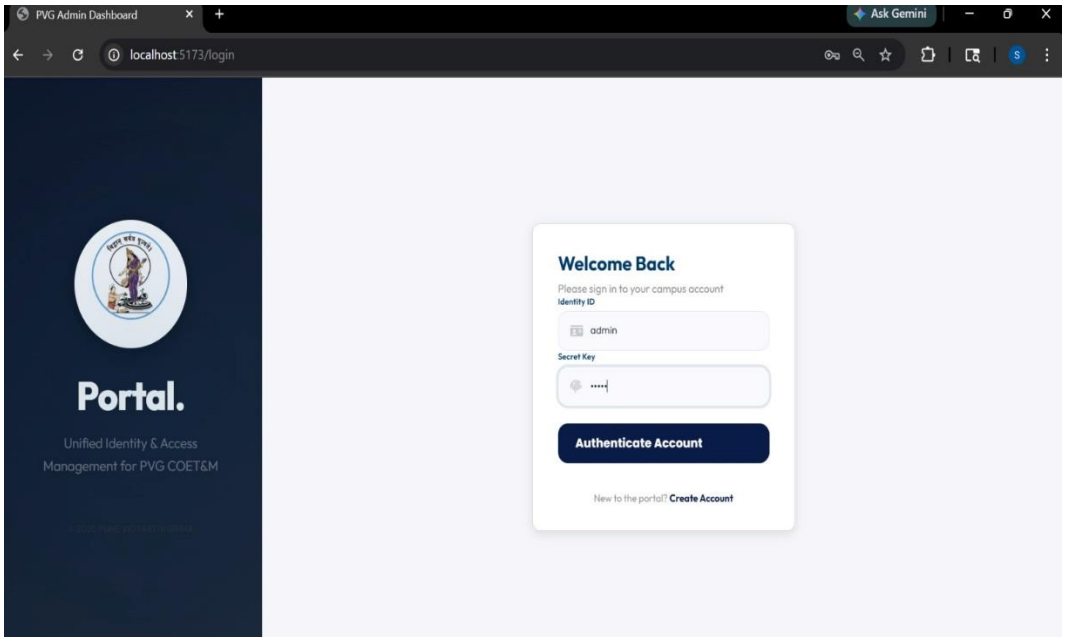
This module ensures that users can only access resources based on their role. For example, a Guest user has limited access, while other roles may have extended permissions. This improves system security and prevents unauthorized access.

5. Welcome

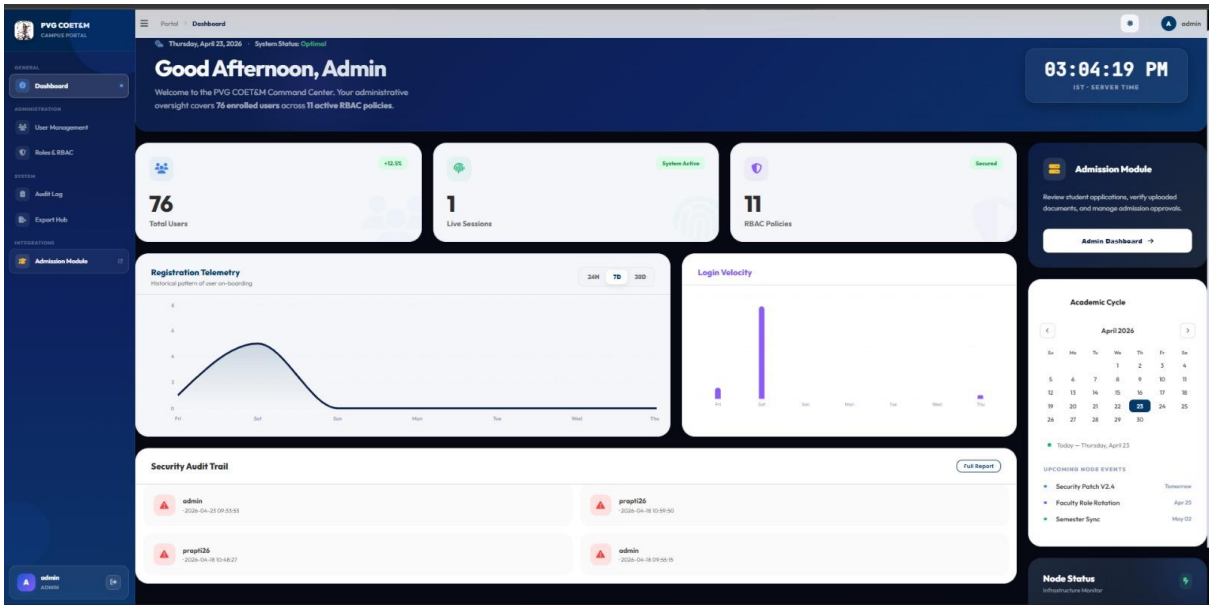
After successful login, the system displays a personalized welcome message to the user. This confirms that authentication is successful and the user has access to the system.

Reports / Output Screens

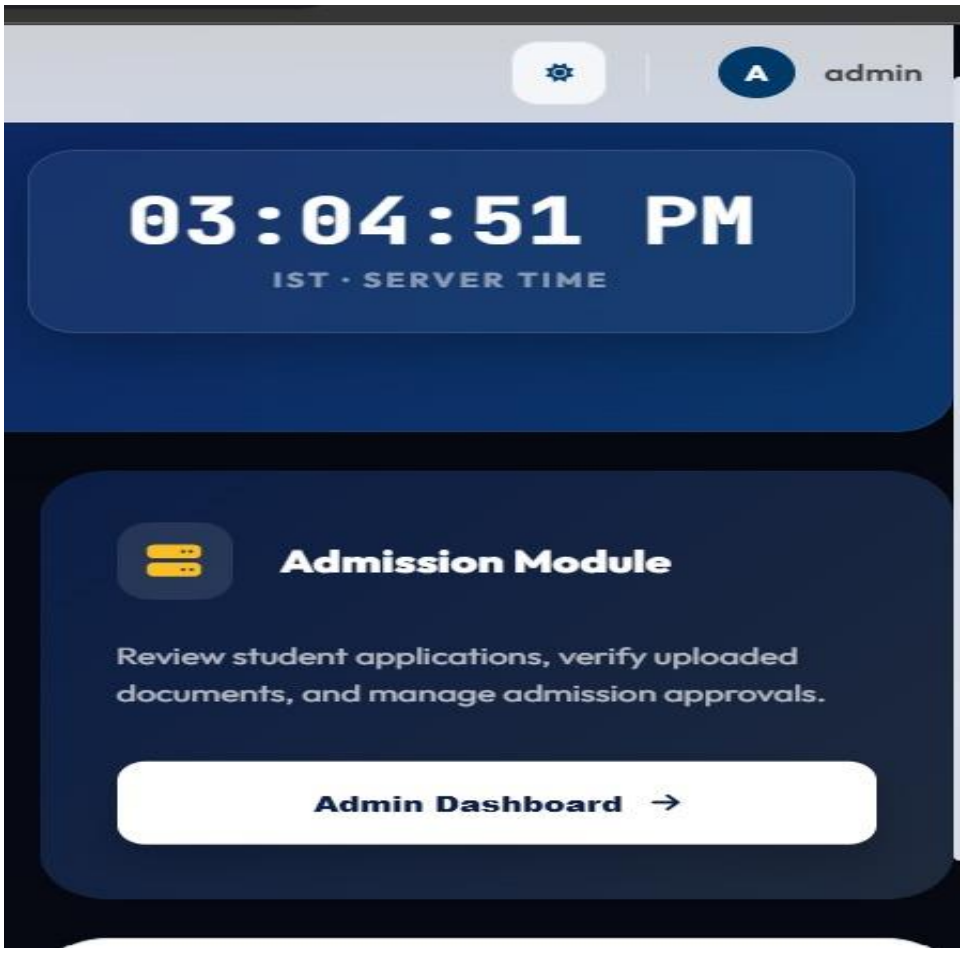
These screenshots show the complete flow of our project from user registration to successful login and access. They help in understanding how the Authentication and Authorization Module works in real-time. Through these output screens, we can clearly see the Signup page, Login process, JWT-based authentication, and the Welcome page after successful login. These screens provide visual proof of the working system and make the project easier to explain and understand.



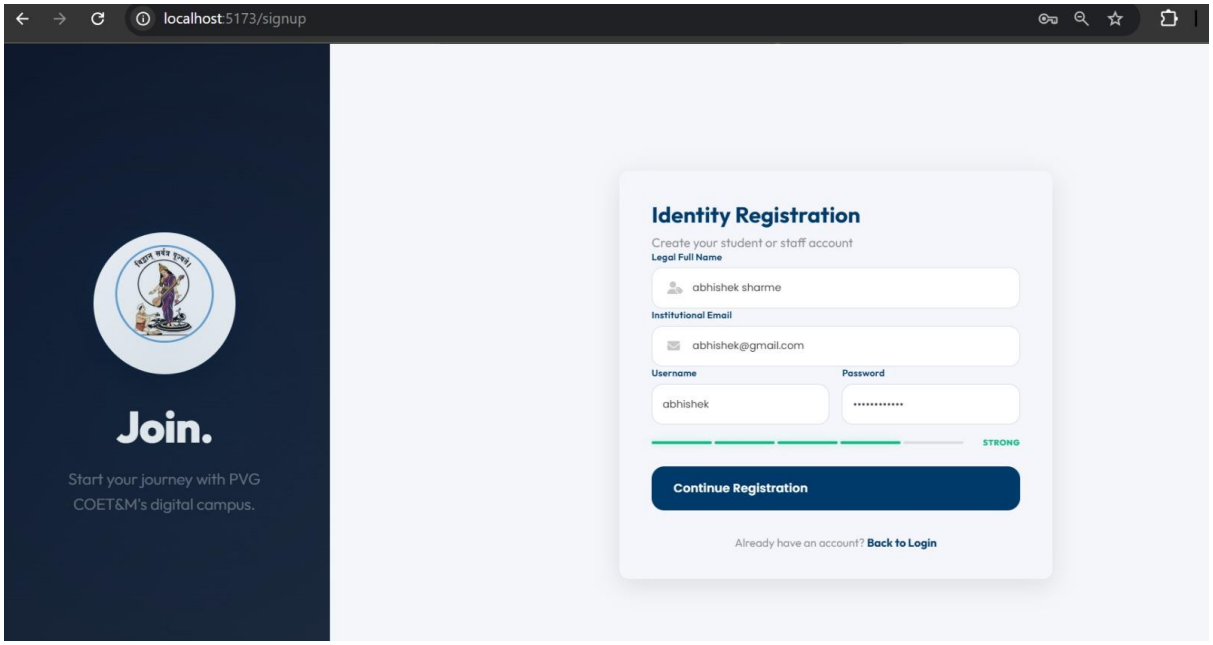
1)Login Portal



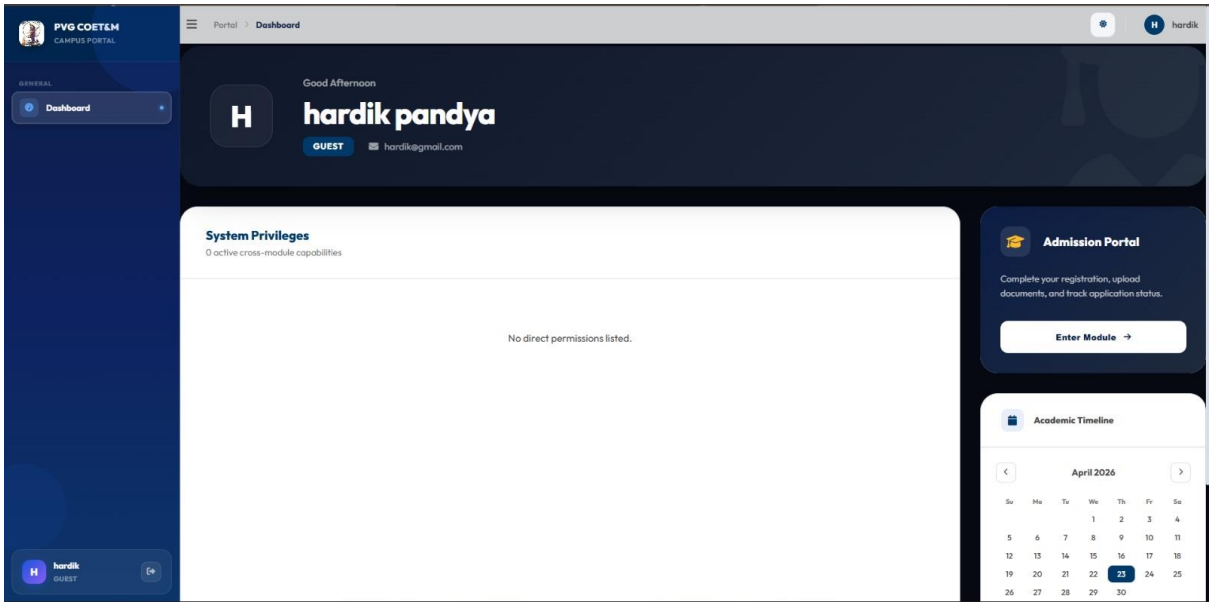
2)Admin Dashboard



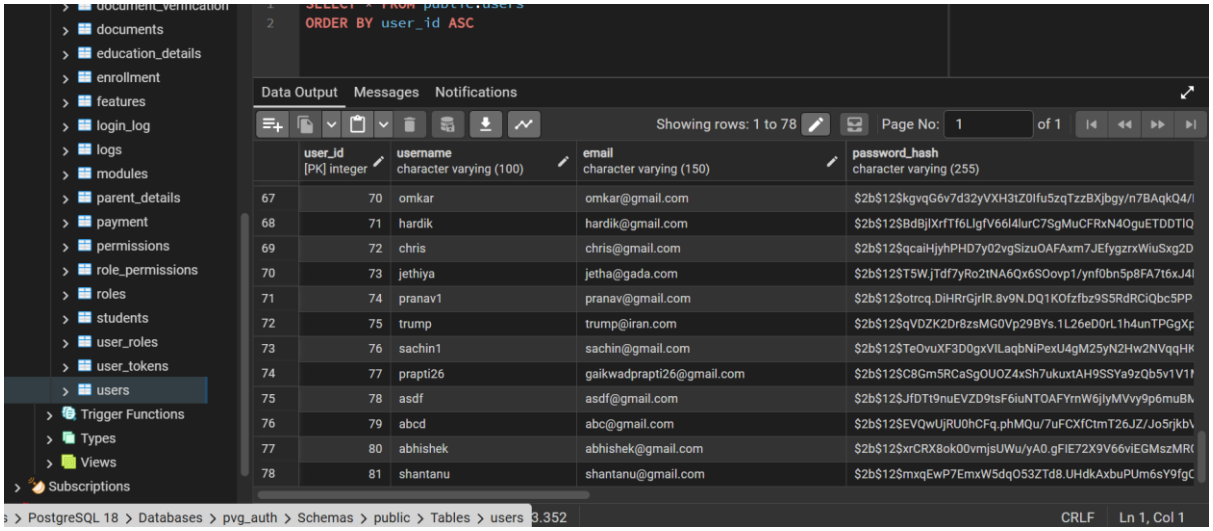
3)link to Admission button



4) New User Sign up page



5)When user login as Guest into portal this is shown.



6)Database and the when user register's its entries get save into user's tables.

Testing & Debugging

• Test Plan

The testing process is carried out to ensure that the system functions correctly, securely, and efficiently. The main objective of testing is to verify that all modules, especially the Authentication and Authorization module, work as expected without errors.

The testing strategy includes both **Black Box Testing** and **White Box Testing**. Black Box Testing is used to validate user inputs and outputs without considering internal code structure, while White Box Testing focuses on verifying internal logic and functionality.

The system is tested under different scenarios such as valid and invalid inputs, login attempts, token validation, and role-based access. The testing ensures that the system meets all functional and non-functional requirements.

• Black Box Testing (Data Validation Test Cases)

Test Case ID	Input	Expected Output	Actual Output	Status
TC01	Valid signup details	User registered successfully	Same as expected	Pass
TC02	Missing required fields	Error message displayed	Same as expected	Pass
TC03	Invalid phone number	Validation error	Same as expected	Pass
TC04	Valid login credentials	Login successful, JWT token generated	Same as expected	Pass
TC05	Invalid username/password	Error message displayed	Same as expected	Pass

Test Case ID	Input	Expected Output	Actual Output	Status
TC06	Empty login fields	Validation error	Same as expected	Pass

• White Box Testing (Functional Test Cases)

Test Case ID	Function	Description	Expected Result	Status
TC07	Password Hashing	Check bcrypt hashing of password	Password stored in encrypted form	Pass
TC08	JWT Generation	Verify token creation on login	Token generated successfully	Pass
TC09	Token Validation	Verify token for protected routes	Access granted with valid token	Pass
TC10	Role Assignment	Check default role assignment	User assigned "Guest" role	Pass
TC11	Token Storage	Check token stored in DB	Token saved correctly	Pass
TC12	Session Handling	Verify token expiry	Session expires after time limit	Pass

• Test Results and Analysis

The testing results show that the system performs as expected under all tested conditions, with all test cases passing successfully, indicating that the authentication and authorization module is functioning correctly. The system effectively validates user inputs, prevents invalid data entry, and securely handles user credentials using bcrypt hashing. JWT tokens are generated and validated properly, ensuring secure session management and controlled access. Role-Based Access Control is implemented efficiently, with users assigned appropriate roles, and the default Guest role ensures minimal access for new users, enhancing security. The system also handles error scenarios by providing clear and user-friendly messages, improving overall usability. During testing, the system remained stable without crashes and handled multiple login attempts consistently. Token expiration and session management worked correctly, preventing unauthorized access after session timeout. Database operations such as storing user and token data were performed accurately, and audit fields were properly maintained for tracking changes. The modular design made it easier to test individual components, and Docker ensured consistent performance across environments. Overall, the system is reliable, secure, and ready for integration with other ERP modules, with scope for future enhancements like multi-factor authentication and advanced logging.

Maintenance

The maintenance phase of the project focuses on keeping the Authentication and Authorization Module secure, updated, and working smoothly after deployment. Regular monitoring is required to check user login activity, token validity, and system performance. JWT tokens and session data must be managed properly to prevent unauthorized access and maintain security. Password protection using bcrypt should be maintained, and database records such as users, roles, and tokens should be updated whenever needed. Audit fields and logs help track system changes and user activities for better control and transparency. Bug fixing, performance improvements, and security updates are important parts of maintenance. As new ERP modules like attendance, fees, exams, hostel, and transport are added, the authentication module should be updated to support smooth integration. Docker and Docker Compose also help in maintaining the system consistently across different environments. Overall, the maintenance phase ensures long-term reliability, scalability, and security of the College ERP system.

Conclusion

In this project, a secure and scalable **Authentication and Authorization module** for a College ERP system was successfully designed and implemented. The system was developed using modern technologies such as React for the frontend, FastAPI for the backend, and PostgreSQL for database management. The module includes essential features like user registration, login, password encryption using bcrypt, and JWT-based authentication for secure session handling.

Role-Based Access Control (RBAC) was implemented to ensure that users can only access data and functionalities based on their assigned roles. By default, new users are assigned a Guest role to maintain minimal access and enhance security. The system also includes token tracking and audit fields to monitor user activity and data changes.

Additionally, the authentication module was successfully integrated with the **Admission Module APIs** using ngrok for secure API exposure. This integration demonstrates the system's ability to connect with other ERP modules and work in a distributed environment. The merged implementation is available in the GitHub repository, showing the combined working of authentication and admission functionalities.

• Key Findings

The project demonstrates that implementing secure authentication mechanisms like JWT and bcrypt significantly improves system security. The use of Role-Based Access Control ensures proper data access management and reduces the risk of unauthorized usage.

The system performed efficiently during testing, handling user authentication, token generation, and validation without errors. The database design with Users, Roles, and Tokens tables provided a structured and scalable way to manage data. The inclusion of audit fields improved transparency and accountability.
Integration with external modules using APIs (via ngrok) showed that the system is flexible and can be extended easily. The use of Docker further simplified deployment and ensured consistency across different environments.
Overall, the project confirms that a modular and secure approach is effective for building a reliable College ERP system. Although the system is functional and secure, several enhancements can be made in future versions. Multi-Factor Authentication (MFA) can be added to provide an additional layer of security during login. A refresh token mechanism can also be implemented to improve session management

Future Enhancement

The system can be extended by adding a fine-grained permission system instead of basic role-based access. Advanced audit logging and monitoring dashboards can be included to track user activities in real time.
UI improvements can be made to enhance user experience, including better design, responsiveness, and accessibility features. Performance optimization can also be considered for handling a large number of users.
Further integration with additional ERP modules such as attendance, fees, examination, hostel, and transport can make the system more comprehensive. Finally, deploying the system on a cloud platform would improve availability and scalability.The future scope of this project is to improve and expand the Authentication and Authorization Module into a complete security foundation for the College ERP system. One major enhancement is adding Multi-Factor Authentication (MFA) using OTP through email or mobile for extra security. A Refresh Token mechanism can also be added to improve session management and allow secure token renewal without repeated login.

The Role-Based Access Control (RBAC) system can be improved by adding fine-grained permissions such as view, edit, delete, approve, and export instead of only broad roles like Guest or Admin. Advanced audit logs and monitoring dashboards can help administrators track login attempts, user activities, and data changes in real time. Since the system uses React, FastAPI, PostgreSQL, and Docker, it is scalable and can support more users and modules in the future. Cloud deployment using AWS or Azure can improve availability, backup, and performance. Database optimization and caching can also improve speed for large-scale usage. Parent and Guardian Portal can also be added. Parents can check attendance, fees, and student performance.

Since the project uses React, FastAPI, PostgreSQL, and Docker, it is highly scalable.It can support more users and future ERP modules easily.
Cloud deployment using AWS or Azure can also be added.This improves backup, availability, and system performance. Database optimization and caching can improve speed for large-scale usage.This helps when the number of users increases in the future.The most important future scope is merging the Authentication Module with all ERP modules.These include Attendance, Fees, Exams, Hostel, Transport, Library, and Placement Cell.
Currently, the system is already integrated with the Admission Module using APIs and ngrok . This proves that the module can connect successfully with other systems.The most important extension is merging the Authentication Module with all ERP modules such as Attendance, Fees, Exams, Hostel, Transport, Library, Placement Cell, and Parent/Guardian Portal. Currently, it is already integrated with the Admission Module using APIs and ngrok. In the future, one login system can provide secure access to all modules, creating a centralized and efficient ERP platform for the entire college.

References

1. FastAPI Official Documentation – Used for learning backend API development, routing, authentication, and FastAPI project structure**.**

2. PostgreSQL Official Documentation – Referred for database design, table creation, queries, and relational database management.

3. React Official Documentation – Used for frontend development, component structure, state management, and UI implementation**.**

4. Docker Official Documentation – Helped in understanding Docker, Docker Compose, and containerized deployment of the project**.**

5. JWT Introduction – Auth0 – Referred for implementing JWT-based authentication, token generation, and secure session handling**.**

6. bcrypt Documentation – PyPI – Used for password hashing and secure storage of user credentials in the database**.**

7. GitHub Repository for Project Integration – Contains the merged Authentication and Admission Module implementation using API integration and ngrok.

8. GitHub Repository for Authentication Module – Contains the complete Authentication and Authorization module with JWT and RBAC implementation.

9. Dribbble – Used for frontend UI design inspiration and modern layout ideas for login and signup pages**.**

10.Figma– Referred for professional UI/UX design inspiration and project presentation ideas**.**

11. Pinterest – Used for simple UI design references, color combinations, and layout inspiration for frontend screens**.**

Student Signature **Guide Signature**

Date: __/__/____